



nextwork.org

Set Up Kubernetes Deployment



yakubu abdullahi yusuf

```
(ec2-user@ip-172-31-36-76 nextwork-flask-backend)$ docker build -t nextwork-flask-backend .
[+] Building 11.9s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 269B
=> [internal] load metadata for docker.io/library/python:3.9-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 42.34kB
=> [1/5] FROM docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> resolve docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> => sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d 15.40MB / 15.40MB
=> => sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c 10.29kB / 10.29kB
=> => sha256:a381ce006821ddce2a0d62e601890f912a9f23d9e1f64007e58328663bd708 1.73kB / 1.73kB
=> => sha256:ac5fec9e2c2244c78acbb322308adf00f8429fcc6aalef08f75572ffdb2124470d 5.18kB / 5.18kB
=> => sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 3.80MB / 3.80MB
=> => sha256:c9008197b8dd9d9b4c7a87e38fa66c8bceb4f7d87df8e9061284b123459f9451 456.92kB / 456.92kB
=> => sha256:9806928fa0da9a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3 247B / 247B
=> => extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b
=> => extracting sha256:c9008197b8dd9d9b4c7a87e38fa66c8bceb4f7d87df8e9061284b123459f9451
=> => extracting sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d
=> => extracting sha256:9806928fa0da9a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3
=> (2/5) WORKDIR /app
=> (3/5) COPY requirements.txt requirements.txt
=> (4/5) RUN pip3 install -r requirements.txt
=> (5/5) COPY . .
```



yakubu abdullahi yusuf

NextWork Student

nextwork.org

Introducing Today's Project!

In this project, I will clone a backend application from GitHub, build a Docker image, and push it to Amazon ECR. I will also troubleshoot installation and configuration issues while gaining hands-on experience with real-world DevOps workflows and exploring the backend code.

Tools and concepts

I used Amazon EKS, Git, ECS, Docker, GitHub, EC2 instances to I used Amazon EKS, Git, ECS, Docker, GitHub, and EC2 instances to push and manage the backend application in a cloud environment. These tools allowed me to version control the code, containerize the application, store and retrieve Docker images.

Project reflection

This project took me approximately thirty to forty minutes, The most challenging part was the permissions and installation of tools on the instance. My favourite part was the version control.

Something new that I learnt from this experience was the management of ECR through the terminal and also through the console.



yakubu abdullahi yusuf

NextWork Student

nextwork.org

What I'm deploying

I'm deploying an app's backend

Next, I retrieved the backend that I plan to deploy. An app's backend means the server-side, which is responsible for handling requests from users, processing, storing and interacting with databases. I retrieved backend code by cloning the project repository from GitHub.

```
Cloning into 'nextwork-flask-backend'...
remote: Enumerating objects: 18, done.
remote: Counting objects: 100% (18/18), done.
remote: Compressing objects: 100% (17/17), done.
remote: Total 18 (delta 4), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (18/18), 6.14 KiB | 6.14 MiB/s, done.
Resolving deltas: 100% (4/4), done.
[ec2-user@ip-172-31-36-76 ~]$ ls
nextwork-flask-backend
[ec2-user@ip-172-31-36-76 ~]$
```



Building a container image

Once I cloned the backend code, my next step is to build a container image of the backend. This is because containerizing the application packages the code and its dependencies together, which the cluster can use in the deployment and it also ensures consistent behavior across different environments and making the application easier to deploy and manage.

When I tried to build a Docker image of the backend, I ran into a permissions error because the commands I ran to install Docker installed it for the root account, which means only the root user can talk to the Docker engine and the default user for the SSH session is ec2-user.

I ran a command to append the ec2-user in to a Docker group which will allow the user to run Docker commands without using sudo all the time and this solves the permissions error. The Docker group is a group in Linux systems that gives users the permission to run Docker commands.



yakubu abdullahi yusuf

NextWork Student

nextwork.org

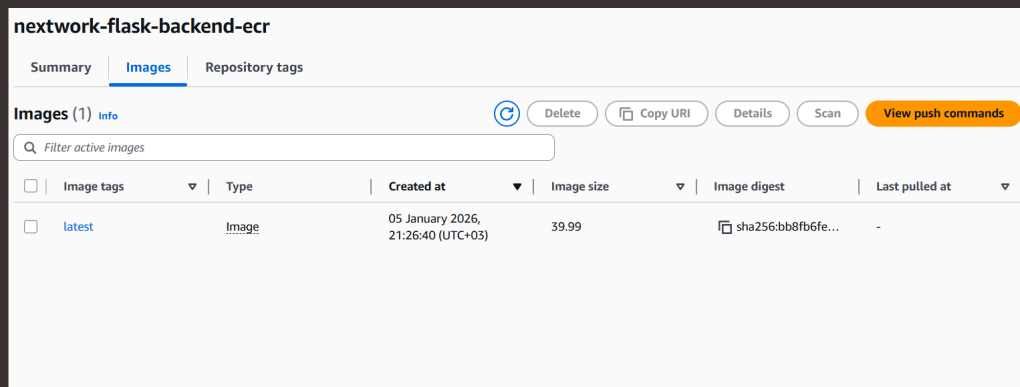
```
[ec2-user@ip-172-31-36-76 nextwork-flask-backend]$ docker build -t nextwork-flask-backend .
[+] Building 11.9s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 269B
=> [internal] load metadata for docker.io/library/python:3.9-alpine
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 42.34kB
=> [1/5] FROM docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> => resolve docker.io/library/python:3.9-alpine@sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c
=> => sha256:127711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d 15.40MB / 15.40MB
=> => sha256:c99b6eb43b3ac4d750db3d6e8b22268d5ea9a99deead7218ce3deda7f2ca029c 10.29kB / 10.29kB
=> => sha256:a381ce006821bdce2a0d62e601890f912e9f23d9e1fb61007e58328863bd7708 1.73kB / 1.73kB
=> => sha256:ac5fec9ec2244c78ecbb822308adf80f8429fc6aa1ef08f75572ffdb2124470d 5.18kB / 5.18kB
=> => sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b 3.80MB / 3.80MB
=> => sha256:c9008197b8dd9d9b4c7a87e38fa66c8bceb4f7d87df8e9061284b123459f9451 456.92kB / 456.92kB
=> => sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3 247B / 247B
=> => extracting sha256:2d35ebdb57d9971fea0cac1582aa78935adf8058b2cc32db163c98822e5dfa1b
=> => extracting sha256:c9008197b8dd9d9b4c7a87e38fa66c8bceb4f7d87df8e9061284b123459f9451
=> => extracting sha256:27711589a568cd76752066b7f5d51377b4eb3d501cf866ff9a2933d7c3db806d
=> => extracting sha256:9806928fa0da94a3d2440c42430d7caa253e6f05e8669881cee924ceecabe3b3
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt requirements.txt
=> [4/5] RUN pip3 install -r requirements.txt
=> [5/5] COPY . .
```



Container Registry

I'm using Amazon ECR in this project to store and manage my Docker images securely in the cloud. ECR is a good choice for the job because it lets Elastic Kubernetes Service (EKS) deploy container images with minimal authentication and configuration setups.

Container registries like Amazon ECR are great for Kubernetes deployment because it keeps deployment simple, EKS can also pull images directly from the registry whenever it needs to start or update containers.





EXTRA: Backend Explained

After reviewing the app's backend code, I've learnt that app's backend handles server-side logic, processes requests, manages the data, and relies on specific dependencies listed in requirements.txt for a smooth operation. The Dockerfile also shows how the backend is containerized for deployment, and exploring the code also helped me understand how the different files work together to make the application function properly.

Unpacking three key backend files

The requirements.txt file lists all the Python packages and their specific versions that the backend application depends on. It ensures that anyone running or deploying the application installs the exact dependencies needed for it to function correctly.

The Dockerfile provides Docker with step-by-step instructions on how to build a container image for the backend application. Key commands in this Dockerfile include FROM python:3.9-alpine, RUN pip3 install -r requirements.txt and so on.

The app.py file contains the main code for your backend, It has three parts which are; Setting up the app and routing, Fetching data and Sending the response.



nextwork.org

The place to learn & showcase your skills

Check out nextwork.org for more projects

